



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

*Go, change the
world*

Machine Learning (MLOps) Tutorial

Topic : Intelligent Workforce Allocation System

Team No: 4

USN

1RV23AI075

1RV23AI078

1RV23AI079

1RV23AI082

Name

Preetham R

Raghavi U Baler

Ramita K A

Rushil Shodavaram



Introduction

- Manual task assignment often ignores employee skills, workload, and availability
- This results in delays, uneven workload distribution, and reduced productivity
- The Intelligent Workforce Allocation System uses machine learning to match employees to tasks
- Task allocation is based on skills, experience, past performance, and availability
- The system improves efficiency through automated, data-driven decisions
- SDG - 8



Motivation

- In many organizations, task allocation is still done manually, which often ignores employee skills, workload balance, and availability.
- This results in inefficiency, employee burnout, and underutilization of talent.
- With growing workforce complexity, there is a strong need for an intelligent, automated system that assigns tasks fairly and efficiently.
- Leveraging machine learning enables organizations to improve productivity, support employee well-being



Problem Definition

Manual task allocation in organizations often fails to consider employee skills, workload, and availability, leading to inefficient task assignments, uneven workload distribution, and reduced overall productivity.



Objectives

- To design an intelligent system for automated workforce task allocation using machine learning techniques.
- To analyze employee skills, experience, availability, workload, and past performance for effective task assignment.
- To improve workforce productivity by ensuring optimal utilization and fair distribution of tasks.
- To reduce manual effort, delays, and bias in the task allocation process.
- To deploy the machine learning model as a scalable service using MLOps practices.
- To monitor model performance and data drift and enable continuous improvement through automated retraining.



Dataset Description

Category	Details
Dataset Type	Structured tabular dataset
Total Records	109,059 records
Total Features	11 attributes
Input Features	Employee data, task data, skill match, availability, performance
Target Variable	suitability_score
ML Task Type	Regression

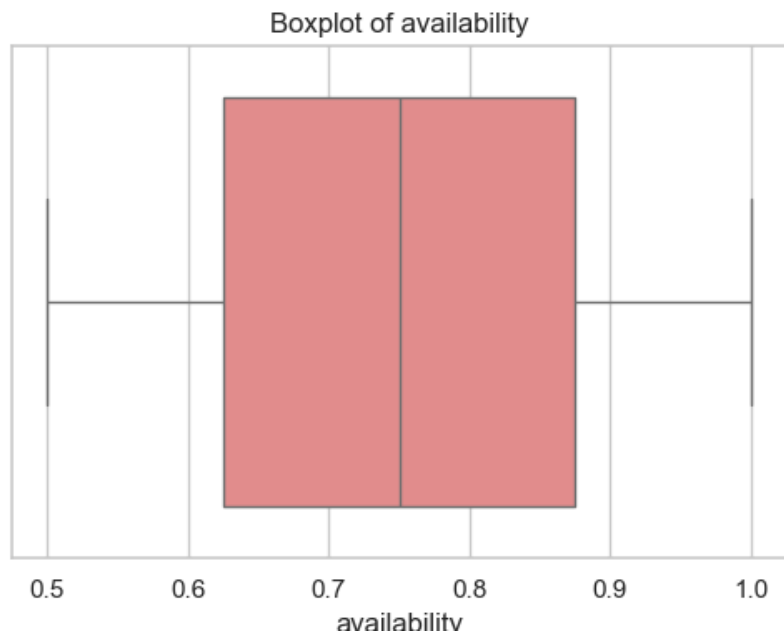
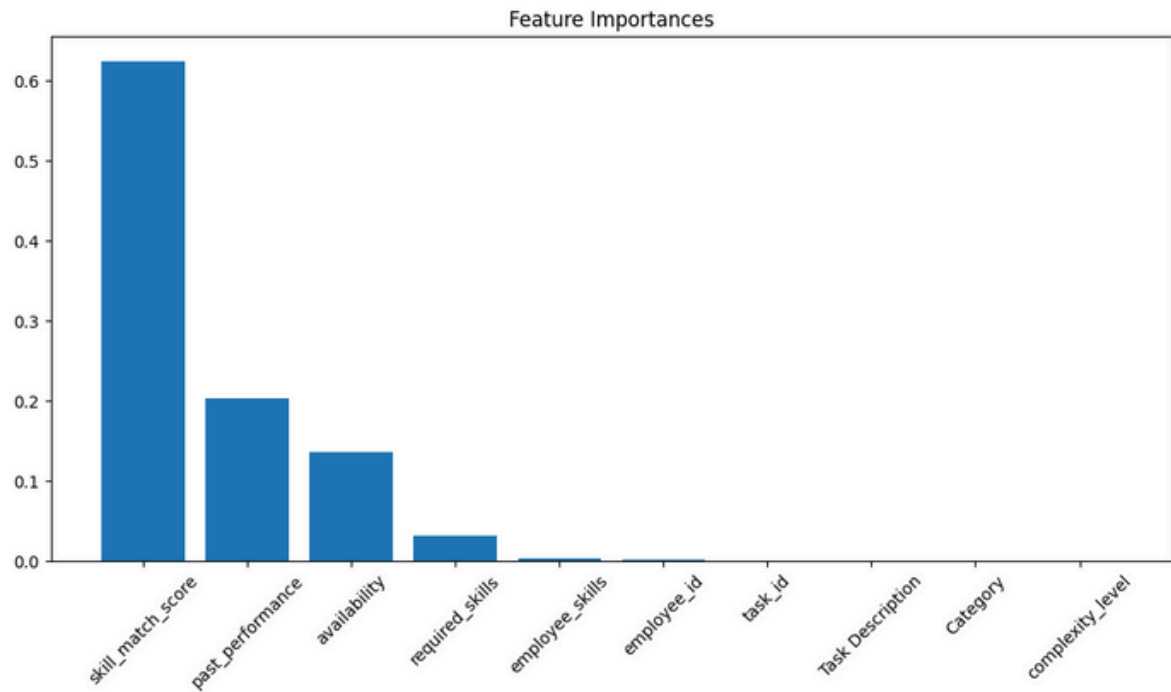


Risk Analysis

Risk	Description	Mitigation Strategy
Data Quality Issues	Incomplete or inaccurate employee or task data may affect	Data cleaning, validation, and preprocessing
Model Bias	Model may favor certain employees due to biased	Balanced dataset and regular model evaluation
Data Drift	Changes in workforce or task patterns over time may reduce	Continuous monitoring and periodic retraining
Deployment Failure	Issues during API or model deployment can disrupt system	Use containerization and testing before deployment
Scalability Issues	System may not handle increased tasks or users efficiently	Scalable deployment using MLOps practices
Security & Privacy	Sensitive employee data may be exposed	Access control, authentication, and secure data handling

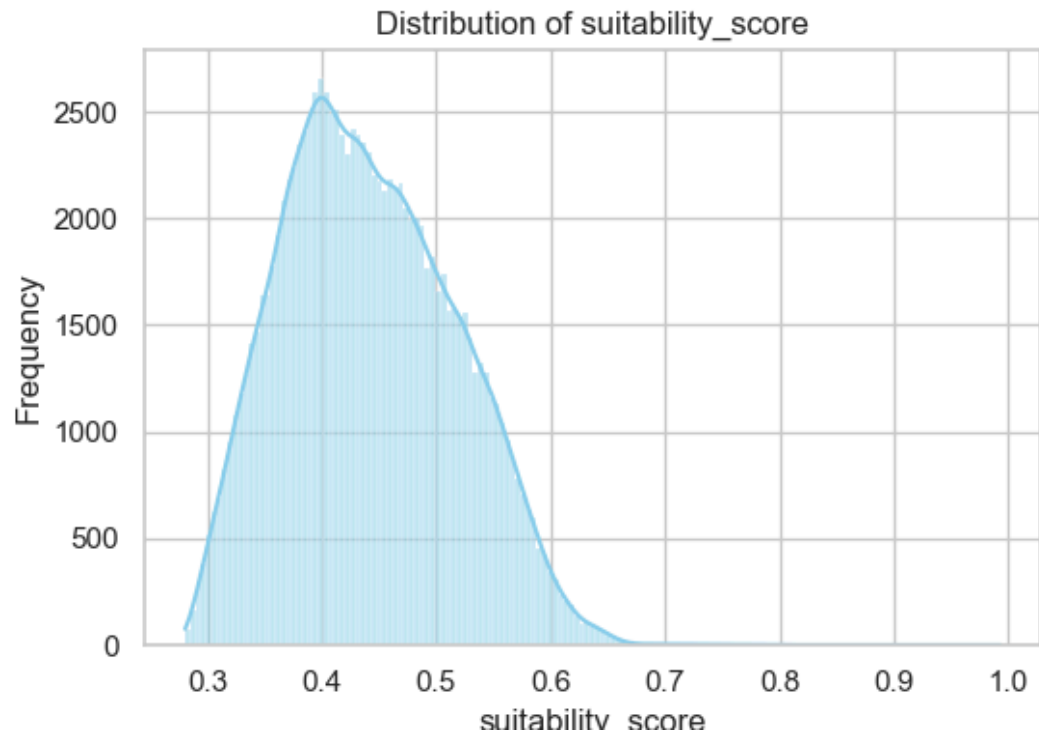
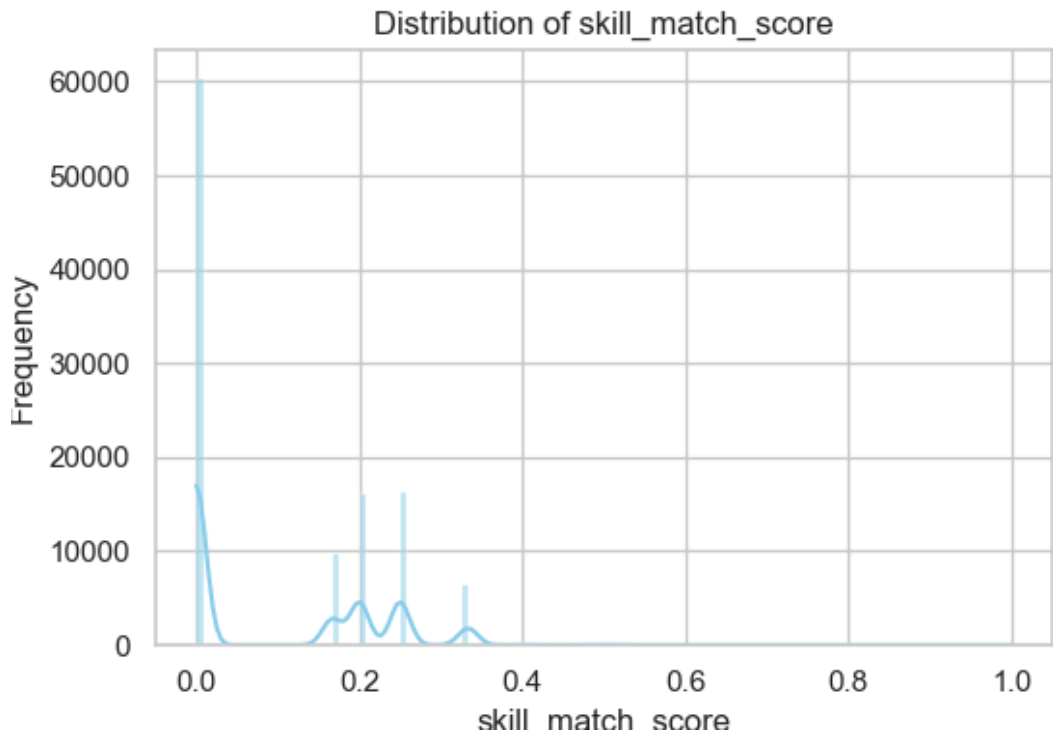


EDA





EDA





Feature Engineering and Feature Selection

- Skill Match Score was computed using Jaccard similarity between employee skills and required task skills, capturing how well an employee fits a task.
- Task Complexity was encoded numerically (Low = 1, Medium = 2, High = 3) to represent task difficulty.
- Availability and Past Performance were normalized to continuous values to reflect real-world workforce variability.
- A Suitability Score was generated as a weighted combination of skill match, availability, and past performance, serving as the target variable.
- Final selected features:
 - skill_match_score
 - availability
 - past_performance
 - complexity_level



ML Algorithms Used

Linear Regression

- Used as a baseline model
- Simple and interpretable
- Works well when feature-target relationships are linear
- Helps explain how each factor affects the suitability score

Random Forest Regressor

- Captures non-linear relationships
- Robust to noise and overfitting
- Handles feature interactions effectively
- Provides feature importance for better interpretability

XGBoost Regressor

- High-performance gradient boosting model
- Learns complex patterns in structured data
- Includes regularization to prevent overfitting
- Achieves the best predictive accuracy

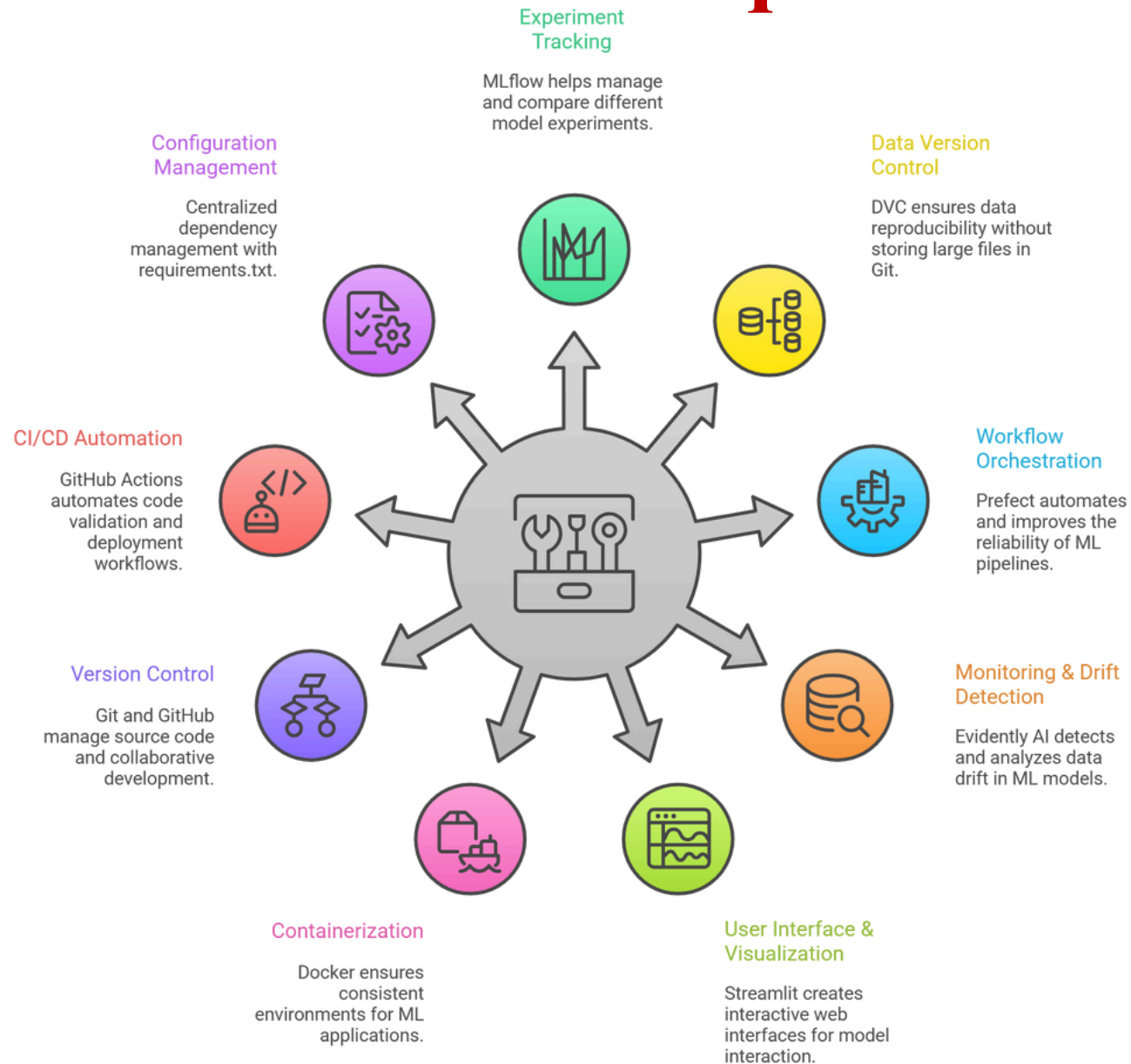


Evaluation of Model

Metric/Model	MAE	R ²
Random Forest	0.010	0.9646
Linear Regression	0.012	0.9604
XGBoost	0.0121	0.9611



Tools and Techniques used





Phase II – MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	DVC
2	Experiment Tracking and Version Control	MLFlow & Github
3	Automation and Pipeline Orchestration	Prefect
4	Model Deployment & Serving	Streamlit
5	Monitoring & Governance and collaboration	EvidentlyAI



MLOPs Tools: DVC

- Tracks dataset versions for the Intelligent Workforce Allocation system (final.csv)
- Maintains history of changes in the training and test datasets
- Ensures each model uses the exact dataset version used for previous runs
- Prevents data leakage across experiments
- Enables reproducible ML experiments, especially when testing model improvements
- Supports collaboration, connecting data versions with MLflow-tracked model versions

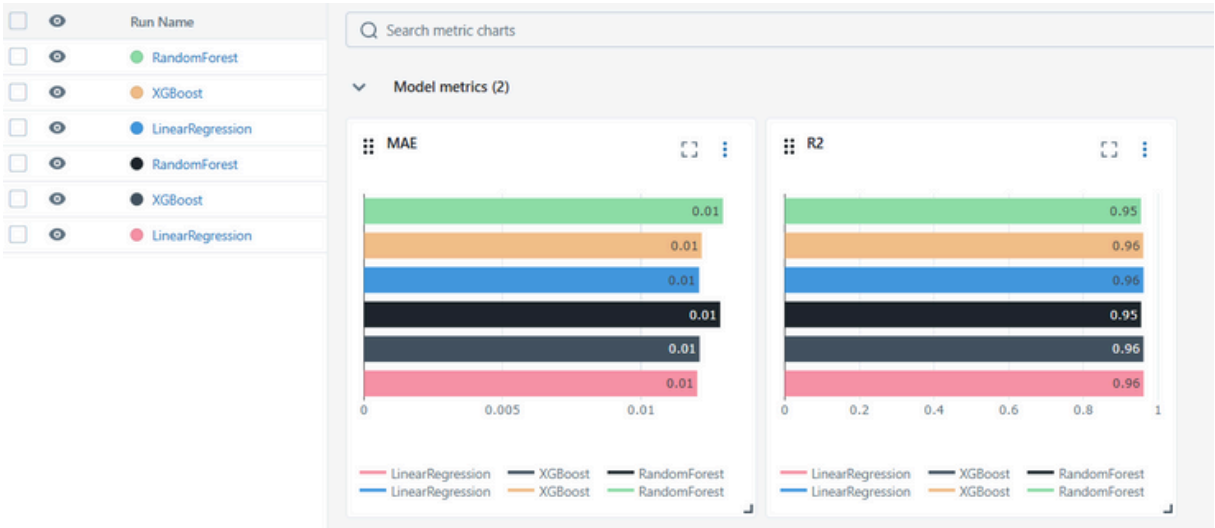


MLOPs Tools:MLflow

- Tracks all experiments for predicting suitability scores of tasks
- Logs model parameters (e.g., n_estimators, max_depth, learning rates)
- Logs metrics such as R^2 , MAE, MSE for Linear Regression, XGBoost, and Random Forest
- Stores trained models as artifacts for version control and later deployment
- Enables easy comparison of model performance:
 - Linear Regression (baseline)
 - XGBoost (boosted tree ensemble)
 - Random Forest (bagged tree ensemble)
- Helps select the best model based on lowest MAE and highest R^2 for task allocation
- Ensures experiment reproducibility and traceability across multiple runs

MLOPs Tools:MLflow

These images describe the MLflow Experiment Tracking and Model Evaluation components of the Intelligent Workforce allocation system, focusing on the performance and history of the allocation models.



Model attributes						No dataset	
Model name	Status	Created ↓	Source run	Registered models	Dataset	MAE	R2
randomforest	Ready	2 days ago	RandomForest	-	-	0.01295672601001543	0.9541703827641977
xgboost	Ready	2 days ago	XGBoost	-	-	0.01219168082172246	0.9604921686627097
linearregression	Ready	2 days ago	LinearRegression	-	-	0.012099997929978085	0.9611249329860654
randomforest	Ready	3 days ago	RandomForest	-	-	0.01285823199391533	0.9546943008788051
xgboost	Ready	3 days ago	XGBoost	-	-	0.01211676879594454	0.9609355267735766
linearregression	Ready	3 days ago	LinearRegression	-	-	0.012028837056553426	0.9614766409358972



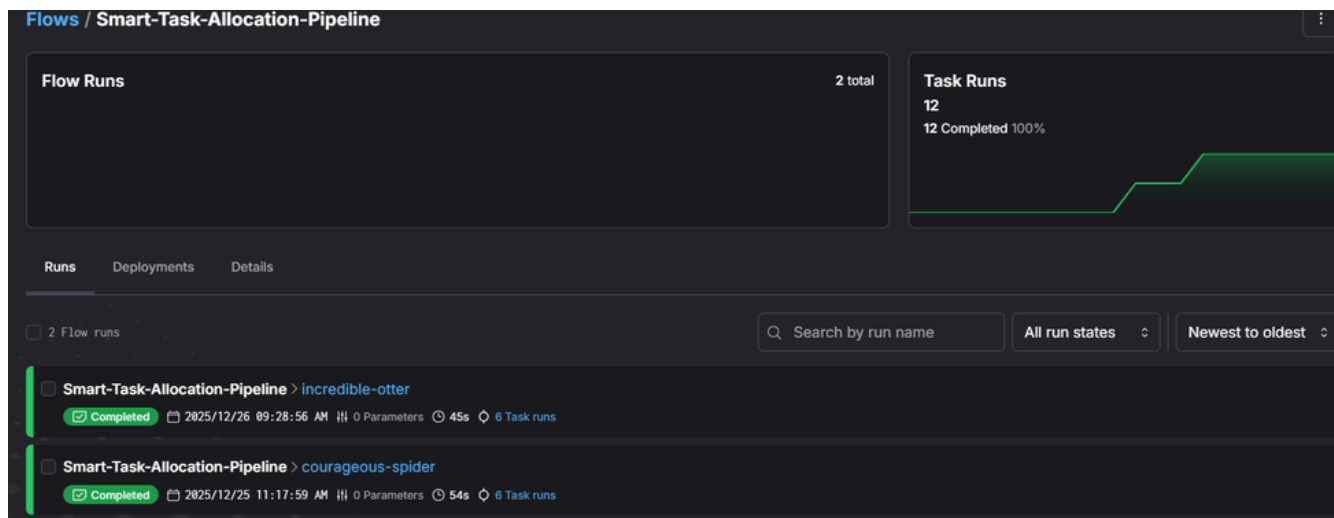
MLOPs Tools: Github

- **Single Source of Truth:** GitHub acts as the central repository for all pipeline-related code and configurations. It ensures everyone works on the same, consistent version of the ML system.
- **Version Control & Change Tracking:** Every modification to code, configurations, or pipeline logic is tracked through commits. This makes it easy to trace what changed, when it changed, and why model behavior evolved.
- **Reproducibility & Auditability:** By tying experiments and deployments to specific Git commits, the pipeline becomes reproducible. Past model results can always be recreated using the exact code state.
- **Collaboration & Quality Control:** GitHub enables structured collaboration using branches, pull requests, and reviews. This reduces errors and enforces quality before changes enter the main pipeline.
- **Pipeline Stability & Reliability:** Controlled merges and versioned releases prevent unstable code from affecting production. This improves the overall reliability of the workforce allocation system.

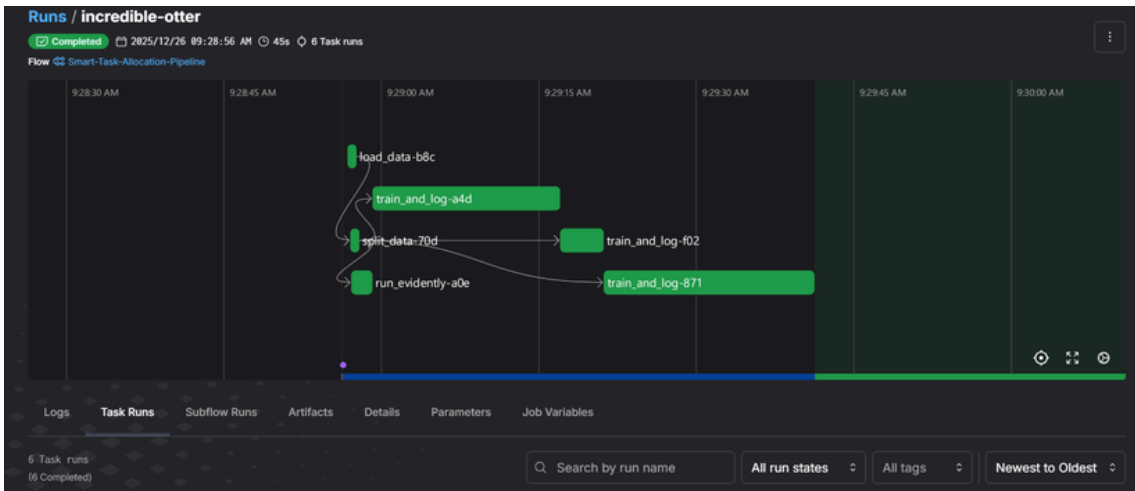


MLOPs Tools: Prefect

- Orchestrates end-to-end workflow:
data loading → preprocessing → model training → evaluation → drift tests
- Automates sequential execution of tasks for all three models
- Handles failures and retries, ensuring robust model training
- Schedules retraining workflows for periodic updates of task suitability predictions
- Ensures pipeline reliability and seamless integration with MLflow logging



MLOPs Tools: Prefect

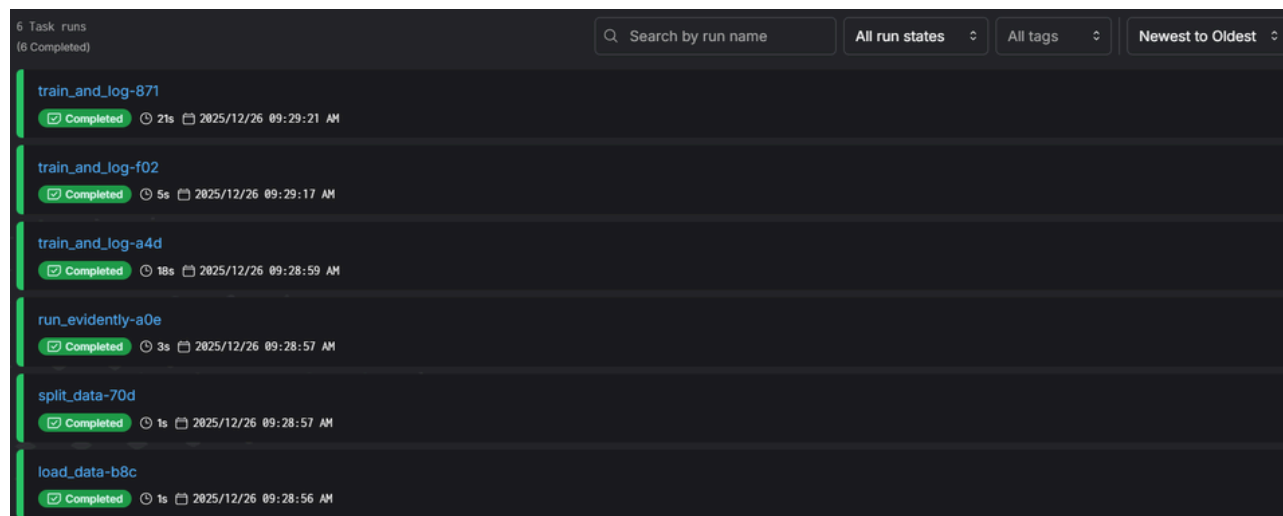


This image typically represents a Flow, which is the high-level "blueprint" or recipe for your task allocation process.

- Role: It acts as the "Manager" of the system. It defines the order of operations—for example, first fetching incoming task requests, then running an allocation algorithm, and finally notifying the assigned resources.

This image typically represents a Flow, which is the high-level "blueprint" or recipe for your task allocation process.

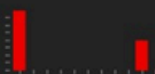
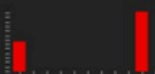
- Role: It acts as the "Manager" of the system. It defines the order of operations—for example, first fetching incoming task requests, then running an allocation algorithm, and finally notifying the assigned resources.



MLOPs Tools: EvidentlyAI

- EvidentlyAI is used to check if our ML model is still working properly after deployment.
- Over time, employee skills, workload, and tasks may change.
- EvidentlyAI helps us detect data drift and performance drop.
- If the model's predictions start becoming inaccurate, it tells us when retraining is needed.
- So, it helps keep the system reliable and accurate in the long run

MLOPs Tools: EvidentlyAI

suitability_score	num			Detected	Wasserstein distance (normed)	2.546563
past_performance	num			Detected	Wasserstein distance (normed)	2.143166
skill_match_score	num			Detected	Wasserstein distance (normed)	0.791107
availability	num			Detected	Wasserstein distance (normed)	0.638313
complexity_inv	num			Detected	Jensen-Shannon distance	0.378157
complexity_level	num			Detected	Jensen-Shannon distance	0.378157

Dataset drift visulatization using EvidentlyAI

MLOPs Tools: Docker

- Docker is used to package our model and code into one container.
- This makes sure the project runs the same on every system.
- It helps avoid deployment issues like missing libraries or version errors.
- Docker also makes the system easy to deploy and scalable.
- Overall, it ensures smooth and safe deployment of our model

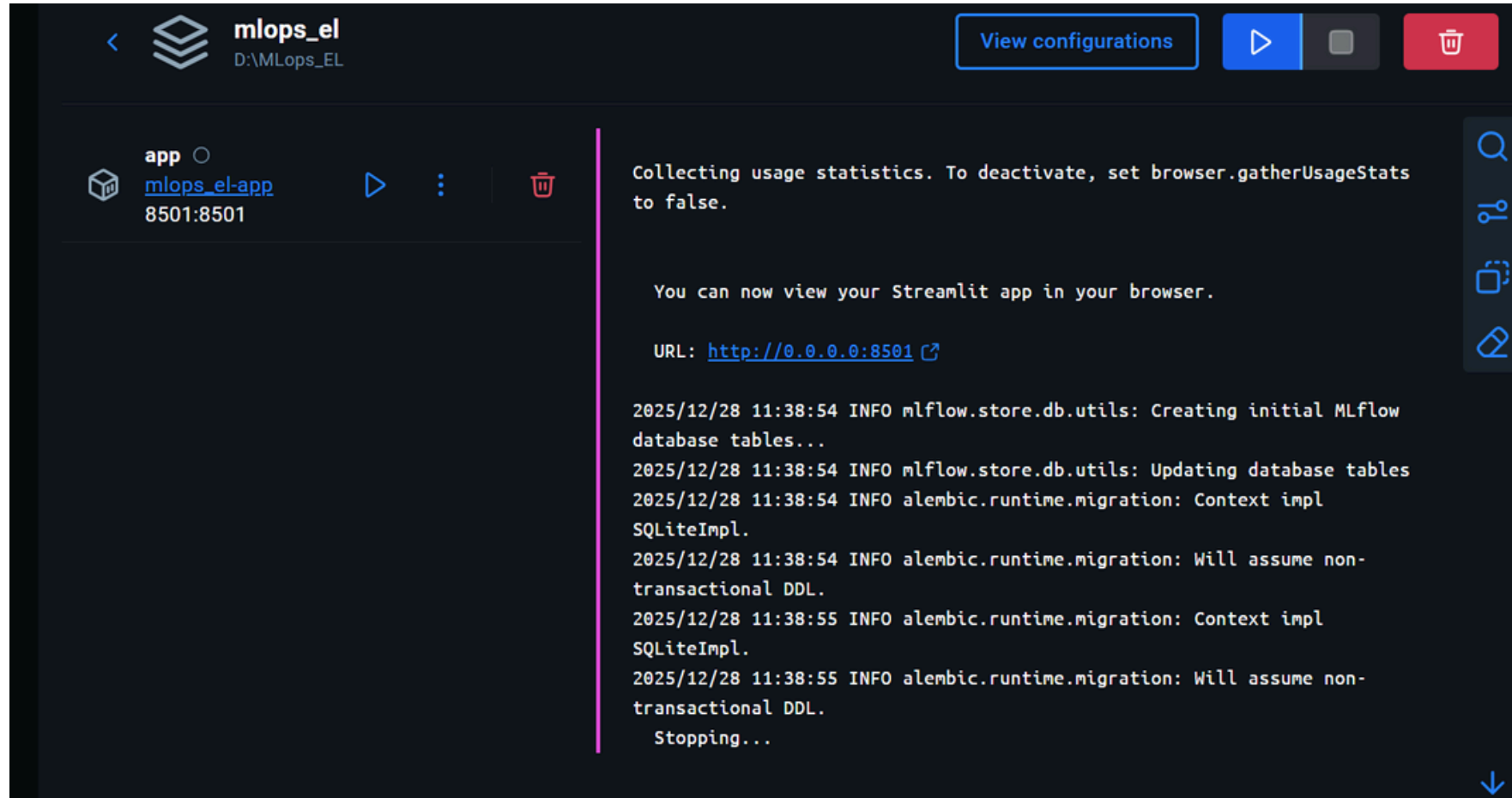


RV College of
Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the
world

MLOPs Tools: Docker





Final Dashboard



Intelligent Workforce Allocation System

Compare predictions from multiple ML models

◆ Input Details

Employee Skills (comma-separated)

python, django, sql

Required Task Skills (comma-separated)

python, flask, sql

Task Complexity

Easy

Availability

0.80

Past Performance

0.90

Deploy



Model Monitoring & Observability

◆

MLflow Model Metrics

	Run Name	MAE	R2	MSE	MAPE	
0	dazzling-hound-470		None	None	None	
1	Random_Forest		0.0126	0.956	0.0003	0.0294
2	XGBoost_Regressor		0.012	0.9619	0.0002	0.028
3	Linear_Regression		0.0119	0.9625	0.0002	0.0278

Evidently Data Drift Report

Drift is detected for 100.0% of columns (6 out of 6).

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
> suitability_score	num			Detected	Wasserstein distance (normed)	2.540563
> past_performance	num			Detected	Wasserstein distance (normed)	2.143186
> skill_match_score	num			Detected	Wasserstein distance (normed)	0.791107
> availability	num			Detected	Wasserstein distance (normed)	0.638813
> complexity_low	num			Detected	Jensen-Shannon distance	0.378157
> complexity_high	num			Detected	Jensen-Shannon distance	0.378157

Drift Alert Status

Data Drift Detected! Immediate attention required.



Conclusion

- Successfully designed and implemented an end-to-end Machine Learning system for smart task allocation.
- Developed and evaluated multiple ML models (Linear Regression, Random Forest, XGBoost) to predict employee-task suitability.
- Integrated MLOps best practices including experiment tracking (MLflow), data versioning (DVC), workflow orchestration (Prefect), and drift monitoring (Evidently AI).
- Built an interactive deployment interface using Streamlit and ensured portability through Docker.
- Implemented CI/CD readiness using GitHub Actions for automated workflows.
- The system ensures reproducibility, scalability, monitoring, and reliability, aligning with real-world industry standards.